

Report on the Adaptive Hedge Algorithm

Gabriele Favazzi Mollica

7 maggio 2026

1 Introduction

In [1], authors showed an adaptive version of the Hedge algorithm, called AdaHedge: It is based on the idea of dividing the time steps into segments; in each segment, the algorithm runs Hedge with a fixed learning rate η , and iteratively compute the cumulative mixability gap Δ for that segment, which is a measure of the gap between the expected loss of the algorithm and the logarithmic mixable loss. More informally, Δ quantifies how difficult the current segment is for the algorithm: if Δ is small, it means that the algorithm is performing well and can continue to use the same learning rate; if Δ is large, it means that the algorithm is struggling and needs to adjust its learning rate to better adapt to the current segment. In particular, when Δ exceeds a certain budget b , the algorithm starts a new segment by decreasing the learning rate η and resetting the learned weights and Δ . The great advantage of this approach is that adaHedge's regret doesn't depend on the time steps T but on the number of segments started (for Lemma 4 in [1]). For Lemma 9 in [1], in case of stochastic losses, the number of segments is bounded by a quantity that doesn't depend on T . Then the regret is $O(K + \log(\frac{1}{\delta})) = O(1)$ with probability at least $1 - \delta$, improving the original $O(\sqrt{T \log K})$ bound for Hedge. However, in the worst case (i.e. the adversarial setting), the number of segments can be as large as $O(\sqrt{T \log K})$, and the regret is $O(\sqrt{T \log K})$ as well, matching the original bound for Hedge.

2 Experiments

In order to empirically evaluate the performance of AdaHedge compared to Hedge with different learning rates, I implemented both algorithms in Python and ran them on synthetic data. Given η^* the optimal learning rate in the worst case scenario, I decided to run Hedge with learning rates $\{\eta^*, 100\eta^*, 10000\eta^*\}$. The second one should be a reasonable learning rate for the tests while the third one is so large that it acts as Follow the Leader (FTL) strategy, at every round putting all the weight on the expert with the lowest cumulative loss so far, which should lead to a very fast adaptation to the losses of the experts, but, in theory, also to a very high regret in case of adversarial losses. The initial learning rate for AdaHedge is set to $\eta_0 = 1.0$ and $\phi = 2.0$ as suggested in [1]. The three settings I considered are:

- **Stochastic setting:** the losses of the experts are generated from a fixed distribution, with one expert having a lower expected loss than the others. In particular the best expert has a loss of 1 with probability 0.3 while the other experts have a loss of 1 with probability 0.5.
- **Adversarial setting:** at every round, the best expert (with $loss = 0$) is randomly chosen among the experts; the other experts have a loss of 1.

- **Low gap setting:** similar to the stochastic setting, but the best expert has a loss of 1 with probability 0.49 while the other experts have a loss of 1 with probability 0.5.

The total number of rounds is $T = 500$ and the number of experts is $K = 20$. The results are averaged over $n_runs = 50$ iterations to reduce the variance of the results.

2.1 Stochastic Setting

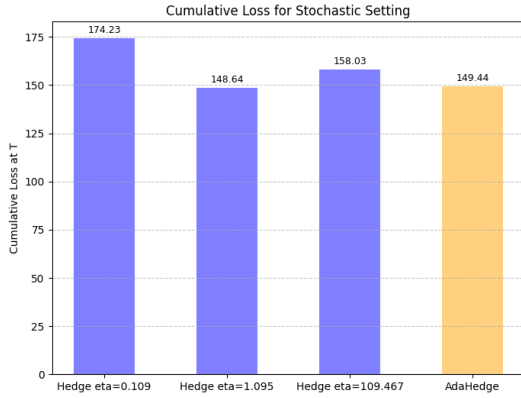


Figure 1: Cumulative Loss (Stochastic)

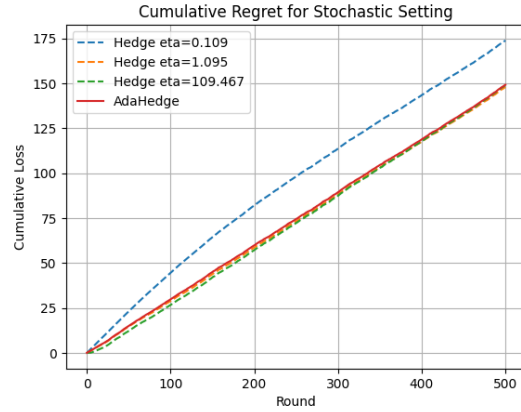


Figure 2: Cumulative Regret (Stochastic)

The results in the stochastic setting are shown in Figure 1 and Figure 2. As we expected, this is the easiest scenario, since AdaHedge started only an average of 0.23 segments. All the algorithms show low cumulative loss, AdaHedge in particular, except for the one with the optimal learning rate η^* : as expected, its regret is high in this easy scenario.

2.2 Adversarial Setting

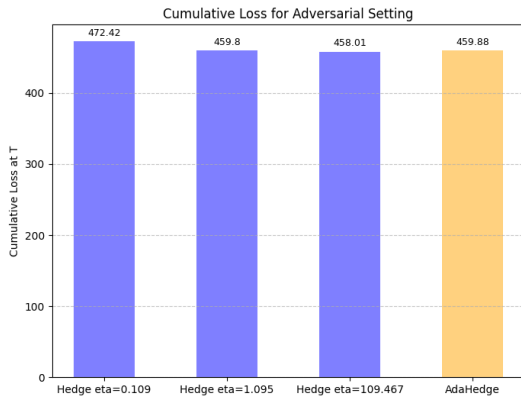


Figure 3: Cumulative Loss (Adversarial)

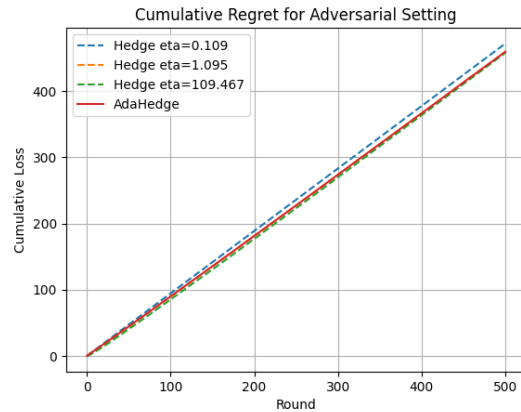


Figure 4: Cumulative Regret (Adversarial)

The results in the adversarial setting are shown in Figure 3 and Figure 4. This scenario is definitely the harder than the previous one, since all the algorithm almost had the worst possible regret, AdaHedge included. AdaHedge started an average of 0.91 segments, witnessing the difficulty of the scenario. However, the algorithm with the optimal learning rate η^* still perform slightly worse than the others.

2.3 Low Gap Setting

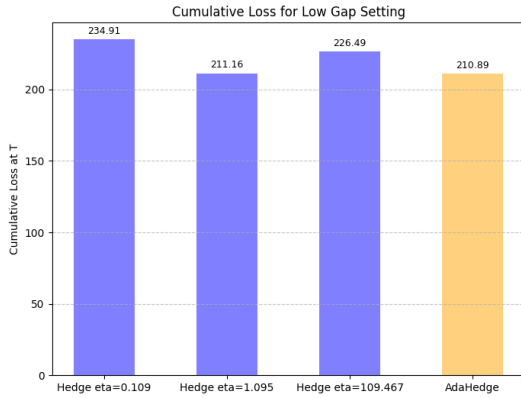


Figura 5: Cumulative Loss (Low Gap)

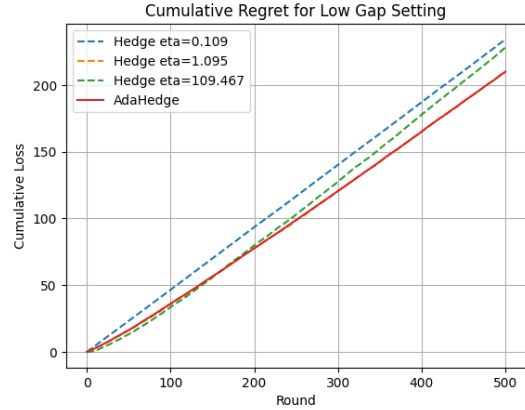


Figura 6: Cumulative Regret (Low Gap)

The results in the low gap setting are shown in Figure 5 and Figure 6. Even this scenario is quite hard, since AdaHedge started an average of 2 segments, and in general algorithms collected worse cumulative loss than in the stochastic setting. The algorithms with the best cumulative loss are AdaHedge and Hedge with $\eta = 100\eta^*$; the one with the optimal learning rate η^* performs again worse than the others, showing that it requires many rounds before performing well. In this scenario the Follow-the-Leader Hedge performed badly, unlike the first scenario, proving that drastically changing the weights distribution leads to a high regret when the gap between the best and the other experts is small, unlike AdaHedge that dynamically adjusts the learning rate and the weights distribution.

Riferimenti bibliografici

- [1] Tim van Erven, Peter Grünwald, Wouter M. Koolen, and Steven de Rooij *Adaptive Hedge*, 2011, <https://arxiv.org/abs/1110.6416>